

HƯỚNG DẪN ĐỌC GIẢ

ẤN BẢN THỨ HAI

CẨM NANG

DevOps

**CÁCH TẠO DỰNG SỰ LINH HOẠT, ĐỘ TIN CẬY
VÀ TÍNH BẢO MẬT ĐẲNG CẤP THẾ GIỚI
TRONG CÁC TỔ CHỨC CÔNG NGHỆ**

GENE KIM · JEZ HUMBLE
PATRICK DEBOIS · JOHN WILLIS

Lời tựa mới và nội dung cập nhật: Nicole Forsgren, PhD
Lời tựa nguyên bản: John Allspaw

BẢN DỊCH TIẾNG VIỆT

Thông tin xuất bản

IT Revolution, 25 NW 23rd Pl, Suite 6314, Portland, OR 97210.

Hướng dẫn dành cho độc giả của The DevOps Handbook, Ấn bản thứ hai © 2021 IT Revolution. *The DevOps Handbook* © 2016 Gene Kim, Jez Humble, Patrick Debois và John Willis. *The DevOps Handbook, Ấn bản thứ hai* © 2020 Gene Kim, Jez Humble, Patrick Debois, John Willis và Nicole Forsgren, PhD.

Mọi quyền được bảo lưu. Để biết thông tin về việc xin phép sao chép các phần trích từ tài liệu hướng dẫn này, vui lòng gửi thư đến: Permissions, IT Revolution Press, LLC, 25 NW 23rd Pl, Suite 6314, Portland, OR 97210.

Bìa và thiết kế: Strauber Design Studio; minh họa bìa: eboy.

Cuốn *The DevOps Handbook* không đi kèm tài liệu hướng dẫn này. Các phiên bản sách giấy, sách điện tử và sách nói được bán tại hầu hết các nhà bán lẻ lớn. Để biết thông tin về ưu đãi đặc biệt khi mua sách số lượng lớn, mời tác giả tham dự sự kiện hoặc tải thêm tài liệu, vui lòng truy cập www.ITRevolution.com.

Tổng quan về *The DevOps Handbook*

Tóm lược

The DevOps Handbook trình bày hệ thống lý thuyết, nguyên tắc và thực hành cần thiết để khởi động thành công một sáng kiến DevOps và đạt được những kết quả mong muốn. Hướng dẫn này dựa trên nhiều thập kỷ lý thuyết quản trị vững chắc; các nghiên cứu về những tổ chức công nghệ có hiệu suất cao; kinh nghiệm của các chuyên gia trong việc hỗ trợ chuyển đổi tổ chức; và các nghiên cứu xác nhận hiệu quả của những thực hành DevOps được đề xuất.

Cuốn sách trình bày lý thuyết và nguyên tắc DevOps thông qua “Ba Phương thức” (*The Three Ways*) - một cách diễn giải cụ thể về lý thuyết nền tảng, lần đầu được giới thiệu trong *The Phoenix Project*. *The DevOps Handbook* dành cho tất cả những ai trực tiếp thực hiện hoặc có ảnh hưởng đến công việc trong dòng giá trị công nghệ - thường gồm Quản lý Sản phẩm, Phát triển, Đảm bảo Chất lượng (QA), Vận hành CNTT và An toàn Thông tin - cũng như lãnh đạo kinh doanh và tiếp thị, nơi khởi nguồn của phần lớn các sáng kiến công nghệ. Cuốn cẩm nang giúp người đọc có kiến thức thực hành về những khái niệm then chốt trong từng lĩnh vực: vừa là tài liệu nhập môn, vừa giới thiệu ngôn ngữ chung để người làm nghề có thể cộng tác với mọi đồng nghiệp trong toàn bộ dòng giá trị CNTT và cùng xác lập các mục tiêu chung.

Trong **Phần I**, sách trình bày ngắn gọn lịch sử DevOps, lý thuyết nền tảng và các chủ đề then chốt từ những hệ tri thức liên quan. Các chương giải thích dòng giá trị; cách DevOps hình thành từ việc áp dụng các nguyên tắc Lean vào dòng giá trị công nghệ; và Ba Phương thức: **Dòng chảy, Phản hồi, Học hỏi và Thử nghiệm liên tục**. Trọng tâm gồm: các nguyên tắc Dòng chảy, giúp đẩy nhanh việc chuyển giao công việc từ Phát triển sang Vận hành rồi đến khách hàng; các nguyên tắc Phản hồi, giúp tổ chức tạo ra những hệ thống làm việc ngày càng an toàn; và các nguyên tắc Học hỏi và Thử nghiệm liên tục, nuôi dưỡng văn hóa tin cậy cao cùng cách tiếp cận khoa học đối với cải tiến tổ chức và việc chấp nhận rủi ro trong công việc hằng ngày.

Trong **Phần II**, sách mô tả cách thức và nơi nên bắt đầu, đồng thời trình bày những khái niệm như dòng giá trị, nguyên tắc và mẫu hình thiết kế tổ chức, mô hình tiếp nhận trong tổ chức và các nghiên cứu tình huống. Một tổ chức nên bắt đầu chuyển đổi DevOps ở đâu? Ai cần tham gia? Các nhóm nên tổ chức đội ngũ, bảo vệ năng lực làm việc và tối đa hóa khả năng thành công ra sao? Các chương trong phần này dẫn dắt toàn bộ quá trình khởi động chuyển đổi DevOps: đánh giá các dòng giá trị của tổ chức, tìm một điểm khởi đầu phù hợp, rồi xây dựng chiến lược để lập một nhóm chuyển đổi chuyên trách với mục tiêu cải tiến cụ thể và kế hoạch mở rộng về sau. Với mỗi dòng giá trị được chuyển đổi, tổ chức phải xác định công việc đang được thực hiện, sau đó xem xét các chiến lược thiết kế và kiểu mẫu tổ chức hỗ trợ tốt nhất cho mục tiêu chuyển đổi. Trọng tâm gồm lựa chọn dòng giá trị để bắt đầu; hiểu công việc trong các dòng giá trị ứng viên; thiết kế tổ chức và kiến trúc có tính đến Định luật Conway; tạo điều kiện đạt kết quả định hướng thị trường nhờ sự hợp tác hiệu quả hơn giữa các chức năng trong toàn dòng giá trị; và bảo vệ, trao quyền cho các nhóm.

Trong **Phần III**, cẩm nang mô tả cách tăng tốc Dòng chảy bằng việc xây dựng nền móng của pipeline triển khai; tạo khả năng kiểm thử tự động nhanh và hiệu quả; tích hợp liên tục; chuyển giao liên tục; và thiết kế kiến trúc cho các đợt phát hành ít rủi ro. Mục tiêu là tạo ra những thực hành kỹ thuật và kiến trúc cần thiết để hình thành và duy trì dòng công việc nhanh từ Phát triển sang Vận hành mà không gây hỗn loạn hay gián đoạn cho môi trường production hoặc khách hàng. Vì vậy, tổ chức phải giảm rủi ro gắn với việc triển khai và phát hành thay đổi vào production. Điều này có thể thực hiện bằng một tập hợp thực hành kỹ thuật gọi là **chuyển giao liên tục** (*continuous delivery*). Trọng tâm gồm các nguyên tắc của chuyển giao liên tục: tạo nền tảng pipeline triển khai; cho phép kiểm thử tự động nhanh và đáng tin cậy; triển khai và thực hành

tích hợp cùng kiểm thử liên tục; tự động hóa, tạo điều kiện và thiết kế kiến trúc cho các đợt phát hành ít rủi ro.

Trong **Phần IV**, sách bàn về cách tăng tốc và khuếch đại Phản hồi: tạo telemetry production hiệu quả để quan sát và giải quyết sự cố; dự đoán vấn đề tốt hơn và đạt mục tiêu; thiết lập phản hồi để Phát triển và Vận hành có thể triển khai thay đổi an toàn; đưa kiểm thử A/B vào công việc hằng ngày; và xây dựng quy trình rà soát, phối hợp nhằm nâng cao chất lượng công việc. Các chương mô tả cách triển khai những thực hành kỹ thuật của Phương thức thứ hai, vốn cần thiết để tạo phản hồi nhanh và liên tục từ Vận hành về Phát triển. Nhờ đó, các vòng phản hồi được rút ngắn và khuếch đại: tổ chức có thể nhìn thấy vấn đề ngay khi chúng xuất hiện và lan truyền thông tin đến mọi người trong dòng giá trị. Các nhóm có thể nhanh chóng phát hiện và khắc phục vấn đề sớm hơn trong vòng đời phát triển phần mềm, lý tưởng là từ rất lâu trước khi chúng gây ra một sự cố thảm họa.

Hơn nữa, tổ chức phải tạo ra một hệ thống làm việc trong đó kiến thức thu được ở khâu sau - tại Vận hành - được tích hợp vào công việc ở khâu trước của Phát triển và Quản lý Sản phẩm. Nhờ vậy, các nhóm có thể nhanh chóng tạo ra cải tiến và bài học, dù nguồn gốc là sự cố production, trục trặc triển khai, dấu hiệu sớm của vấn đề hay mẫu hình sử dụng của khách hàng. Đồng thời, cần có một quy trình giúp mọi người nhận phản hồi về công việc của mình, làm cho thông tin trở nên hữu hình để phục vụ học hỏi, và cho phép kiểm nghiệm nhanh các giả thuyết sản phẩm nhằm xác định liệu tính năng đang xây dựng có thực sự giúp đạt mục tiêu tổ chức hay không. Các chương này cũng chỉ ra cách tạo telemetry từ quá trình build, test và deploy; từ hành vi người dùng; từ sự cố và gián đoạn production; cũng như từ vấn đề kiểm toán và vi phạm an ninh. Trọng tâm gồm tạo telemetry để nhìn thấy và giải quyết vấn đề; dùng telemetry để dự đoán vấn đề tốt hơn và đạt mục tiêu; tích hợp nghiên cứu cùng phản hồi người dùng vào công việc của nhóm sản phẩm; tạo phản hồi để Phát triển và Vận hành triển khai an toàn; và dùng phản hồi nhằm nâng cao chất lượng thông qua rà soát đồng cấp và lập trình cặp.

Trong **Phần V**, sách mô tả cách tăng tốc Học hỏi liên tục bằng việc xây dựng văn hóa công bằng, chuyển các phát hiện cục bộ thành cải tiến toàn cục và dành thời gian hợp lý cho học hỏi, cải tiến tổ chức. Các thực hành được trình bày tạo cơ hội học hỏi sớm nhất, nhanh nhất, thường xuyên nhất và ít tốn kém nhất có thể. Chúng bao gồm việc rút ra bài học từ tai nạn và thất bại - những điều không thể tránh khỏi khi làm việc với hệ thống phức tạp - cũng như tổ chức và thiết kế hệ thống làm việc để tổ chức luôn thử nghiệm, học hỏi và liên tục làm cho hệ thống an toàn hơn. Kết quả là khả năng chống chịu cao hơn và kho tri thức tập thể ngày càng phong phú về cách hệ thống thực sự vận hành, nhờ đó các nhóm có năng lực đạt mục tiêu tốt hơn. Những chương tiếp theo thể chế hóa các nghi thức giúp tăng an toàn, cải tiến và học hỏi liên tục: xây dựng văn hóa công bằng để tạo điều kiện an toàn; chủ động đưa lỗi vào production để rèn khả năng chống chịu; biến phát hiện cục bộ thành cải tiến toàn cục; và dành thời gian cho cải tiến, học hỏi tổ chức.

Cuối cùng, **Phần VI** mô tả cách tích hợp đúng đắn bảo mật và tuân thủ vào công việc hằng ngày: đưa các kiểm soát bảo mật phòng ngừa vào kho mã nguồn và dịch vụ dùng chung; tích hợp bảo mật vào pipeline triển khai; tăng cường telemetry để phát hiện và khôi phục tốt hơn; bảo vệ pipeline triển khai; và đặt các mục tiêu quản lý thay đổi. Những hoạt động đã mô tả được mở rộng để không chỉ đáp ứng mục tiêu của Phát triển và Vận hành mà còn đồng thời đạt mục tiêu An toàn Thông tin, tạo mức đảm bảo cao về tính bí mật, toàn vẹn và sẵn sàng của dịch vụ và dữ liệu.

Thay vì đợi đến cuối quy trình mới kiểm tra rồi “nhét” bảo mật vào sản phẩm, các biện pháp kiểm soát bảo mật phải được tạo ra và tích hợp vào công việc hằng ngày của Phát triển và Vận hành. Lý tưởng nhất là công việc này được tự động hóa và đưa vào pipeline triển khai. Các thực hành thủ công, bước chấp nhận và quy trình phê duyệt cũng cần được hỗ trợ bằng kiểm soát tự động, giảm phụ thuộc vào những biện pháp như phân tách nhiệm vụ và quy trình phê duyệt thay đổi. Khi tự động hóa các hoạt động này, tổ chức có thể tạo bằng chứng theo yêu cầu để chứng minh rằng các kiểm soát đang vận hành hiệu quả. Sau cùng, cách làm này nâng cao bảo mật, tạo quy trình dễ kiểm toán hơn và chứng thực được hiệu quả kiểm

soát nhằm đáp ứng nghĩa vụ pháp lý và hợp đồng. Điều đó được thực hiện bằng cách biến bảo mật thành công việc của mọi người; tích hợp kiểm soát phòng ngừa vào kho mã nguồn dùng chung; tích hợp bảo mật với pipeline triển khai và telemetry để phát hiện, phục hồi tốt hơn; bảo vệ pipeline triển khai; gắn hoạt động triển khai với quy trình phê duyệt thay đổi; và giảm phụ thuộc vào việc phân tách nhiệm vụ.

Chú giải của người dịch - Ba Phương thức: Có thể hình dung như ba vòng tăng cường lẫn nhau: (1) làm cho công việc đi từ ý tưởng đến khách hàng nhanh và trơn tru; (2) đưa tín hiệu từ khâu sau quay về khâu trước càng sớm càng tốt; (3) dành năng lực để thử nghiệm, học và cải tiến cả hệ thống. Ví dụ, một lỗi production không chỉ được sửa; tín hiệu lỗi phải quay về bộ kiểm thử, và bài học phải trở thành cải tiến dùng chung.

Phân tích theo từng chương

Phần mở đầu

Tóm lược

Thông thường, Phát triển và Vận hành CNTT ở thế đối đầu; hoạt động kiểm thử và An toàn Thông tin diễn ra quá muộn để sửa các vấn đề được phát hiện; gần như mọi hoạt động quan trọng đều đòi hỏi quá nhiều thao tác thủ công và quá nhiều lần bàn giao. Hệ quả là thời gian hoàn tất công việc cực kỳ dài, chất lượng công việc đầy vấn đề và hỗn loạn, tác động xấu đến khách hàng và hoạt động kinh doanh. Các tổ chức cần thay đổi cách làm việc; DevOps chỉ ra con đường tốt nhất để tiến lên.

Với DevOps, chủ sở hữu sản phẩm, Phát triển, QA, Vận hành CNTT và An toàn Thông tin cùng hướng đến một mục tiêu chung: cho phép công việc đã hoạch định chảy nhanh vào production, đồng thời đạt độ ổn định, tin cậy, sẵn sàng và bảo mật đẳng cấp thế giới. Các nhóm liên chức năng kiểm nghiệm nghiêm ngặt giả thuyết về những tính năng có thể làm người dùng hài lòng nhất và thúc đẩy mục tiêu tổ chức. Họ quan tâm đến việc triển khai tính năng cho người dùng, đồng thời chủ động bảo đảm công việc chảy trơn tru, thường xuyên qua toàn bộ dòng giá trị mà không gây hỗn loạn hay gián đoạn cho Vận hành CNTT hoặc bất kỳ khách hàng nội bộ hay bên ngoài nào.

Đồng thời, QA, Vận hành CNTT và An toàn Thông tin luôn tìm cách giảm ma sát cho nhóm, tạo ra hệ thống làm việc giúp lập trình viên năng suất hơn và đạt kết quả tốt hơn. Nhờ đó, tổ chức có thể tạo một hệ thống làm việc an toàn, nơi các nhóm nhỏ phát triển, kiểm thử và triển khai mã nhanh chóng, độc lập; chuyển giao giá trị tới khách hàng một cách nhanh, an toàn, bảo mật và đáng tin cậy. Điều này giúp tối đa hóa năng suất của lập trình viên, tạo điều kiện học hỏi trong tổ chức, nâng cao sự hài lòng của nhân viên và giành lợi thế trên thị trường.

Chủ đề

1. Xung đột cố hữu giữa Phát triển và Vận hành CNTT tạo nên một vòng xoáy đi xuống.
2. DevOps có thể phá vỡ vòng xoáy này.
3. Có bằng chứng rõ ràng về giá trị kinh doanh của DevOps.

Câu hỏi thảo luận

1. Một số mục tiêu cạnh tranh lẫn nhau của Phát triển và Vận hành CNTT là gì?
2. “Vòng xoáy đi xuống” là gì, và tại sao nó xảy ra ở khắp nơi, trong các tổ chức thuộc mọi loại hình và quy mô?
3. Các tổ chức cung cấp sản phẩm và dịch vụ công nghệ có thể học gì từ cuộc cách mạng sản xuất của thập niên 1980?

Đọc thêm

- *Vượt lên trên mục tiêu: Eliyahu Goldratt bàn về Lý thuyết các điểm ràng buộc (Beyond the Goal)*, Eliyahu M. Goldratt.
- “CEO GE Jeff Immelt: Hãy chấm dứt cuộc tranh luận liệu chúng ta có đang ở trong bong bóng công nghệ” (*GE CEO Jeff Immelt: Let’s Finally End the Debate over Whether We Are in a Tech Bubble*), Jeff Immelt, *Business Insider*.
- *Lợi thế tốc độ cao: Cách các công ty dẫn đầu thị trường tận dụng sự xuất sắc trong vận hành để đánh bại đối thủ (The High-Velocity Edge)*, Steven J. Spear.

- *Thánh-người huyền thoại: Các tiểu luận về kỹ nghệ phần mềm (The Mythical Man-Month)*, ấn bản kỷ niệm, Frederick P. Brooks, Jr.

Chương 1

Tóm lược

Chương này giới thiệu lý thuyết nền tảng của Sản xuất tinh gọn (*Lean Manufacturing*) cùng Ba Phương thức - các nguyên tắc mà từ đó có thể suy ra mọi hành vi DevOps quan sát được. Trọng tâm chủ yếu là lý thuyết và nguyên tắc, mô tả nhiều thập kỷ bài học từ sản xuất, các tổ chức có độ tin cậy cao và mô hình quản trị có mức tin cậy cao - những nguồn hình thành nên thực hành DevOps. Các nguyên tắc, mẫu hình cụ thể và cách áp dụng thực tế vào dòng giá trị công nghệ được trình bày trong những chương còn lại.

Chương này mô tả khái niệm dòng giá trị; thời gian từ lúc bắt đầu đến lúc hoàn tất (*lead time*) như một thước đo then chốt về hiệu quả của cả dòng giá trị sản xuất lẫn công nghệ; và các khái niệm cấp cao của từng Phương thức trong Ba Phương thức - nền tảng của DevOps. Ba Phương thức là Dòng chảy, Phản hồi, và Học hỏi và Thử nghiệm liên tục.

Chủ đề

1. Dòng giá trị là khái niệm nền tảng của Lean có thể áp dụng cho công nghệ.
2. Lead time là thước đo quan trọng về hiệu quả của dòng giá trị công nghệ.
3. Ba Phương thức là nền tảng của DevOps.

Câu hỏi thảo luận

1. Vì giá trị chỉ được tạo ra khi dịch vụ đang chạy trên production, tổ chức làm thế nào để vừa bảo đảm dòng chuyển giao nhanh, vừa triển khai mà không gây hỗn loạn và gián đoạn?
2. Trong trạng thái DevOps lý tưởng, lập trình viên nhận phản hồi nhanh và liên tục về công việc. Loại phản hồi này cho phép họ làm gì với mã của mình?
3. Tổ chức có thể làm gì để tạo môi trường cho Ba Phương thức của DevOps phát triển?
4. * Hãy thảo luận giá trị và cách dùng các chỉ số dòng chảy; chúng có thể hoặc không thể hỗ trợ tổ chức của bạn lập bản đồ dòng giá trị như thế nào?
5. * Trong Phần 1 của nghiên cứu tình huống American Airlines, Maya Leibman trình bày hành trình DevOps ba năm. Bạn và tổ chức có thể rút ra những bài học then chốt nào?

Đọc thêm

- “Mười hai nguyên tắc của Phần mềm Agile” (*Twelve Principles of Agile Software*), Kent Beck và cộng sự, AgileManifesto.org.
- *Toyota Kata: Quản lý con người để cải tiến, thích ứng và đạt kết quả vượt trội*, Mike Rother.
- *Lập bản đồ dòng giá trị: Cách trực quan hóa công việc và đồng bộ lãnh đạo cho chuyển đổi tổ chức*, Karen Martin và Mike Osterling.
- *Từ dự án đến sản phẩm: Cách tồn tại và phát triển trong thời đại gián đoạn số với Flow Framework*, Mik Kersten.

Video nên xem

- “DevOps: Tiến gần độ cao hành trình” (*DevOps: Approaching Cruising Altitude*), DevOps Enterprise Summit, Las Vegas, 2020: <https://videolibrary.doesvirtual.com/?video=550704282>

Chú giải của người dịch - lead time: Đây là tổng thời gian từ lúc có yêu cầu đến lúc giá trị đến tay người dùng, không chỉ là thời gian kỹ sư thực sự viết mã. Ví dụ, một thay đổi chỉ mất hai giờ để lập trình nhưng chờ phê duyệt và xếp lịch mười ngày thì lead time vẫn xấp xỉ mười ngày.

Chương 2

Tóm lược

Trong dòng giá trị công nghệ, công việc thường chảy từ Phát triển sang Vận hành - các khu vực chức năng nằm giữa doanh nghiệp và khách hàng. Phương thức thứ nhất đòi hỏi dòng công việc nhanh và trơn tru từ Phát triển sang Vận hành để sớm chuyển giao giá trị cho khách hàng. Tổ chức tối ưu cho mục tiêu toàn cục này thay vì các mục tiêu cục bộ, đồng thời tăng dòng chảy bằng cách làm cho công việc trở nên hữu hình, giảm kích thước lô và khoảng cách giữa các đợt công việc, và tích hợp chất lượng ngay từ đầu để ngăn lỗi bị chuyển xuống các trung tâm công việc phía sau.

Bằng cách tăng tốc dòng chảy qua dòng giá trị công nghệ, tổ chức có thể giảm lead time để đáp ứng yêu cầu của khách hàng nội bộ và bên ngoài, đồng thời nâng cao chất lượng công việc và trở nên linh hoạt hơn. Mục tiêu là giảm thời gian cần thiết để đưa thay đổi vào production và tăng độ tin cậy, chất lượng của dịch vụ. Cải thiện dòng chảy qua dòng giá trị công nghệ là điều thiết yếu để đạt kết quả DevOps. Ta có thể thực hiện bằng cách làm cho công việc hữu hình; giới hạn công việc đang thực hiện; giảm kích thước lô và số lần bàn giao; liên tục xác định, đánh giá các điểm ràng buộc; và loại bỏ những trở ngại trong công việc hằng ngày.

Chủ đề

1. Muốn thấy nơi công việc đang chảy tốt và nơi đang xếp hàng hoặc đình trệ, cần làm cho công việc hữu hình nhất có thể.
2. Liên tục xác định và nâng cấp các điểm ràng buộc.
3. Loại bỏ trở ngại và lãng phí trong dòng giá trị.

Câu hỏi thảo luận

1. Tổ chức có thể làm gì để tăng dòng công việc nhanh, trơn tru từ Phát triển sang Vận hành nhằm sớm chuyển giao giá trị cho khách hàng?
2. Để bảo đảm công việc chảy tốt và phát hiện nơi nó đình trệ, tổ chức có thể dùng những phương pháp nào để làm công việc hữu hình nhất có thể?
3. “Năm bước tập trung” mà Tiến sĩ Eliyahu M. Goldratt đề xuất để giải quyết một trung tâm công việc bị thiếu đầu vào, nằm sau một nút thắt cổ chai, là gì?

Đọc thêm

- *Tư duy tinh gọn: Xóa bỏ lãng phí và tạo dựng thịnh vượng trong doanh nghiệp*, James P. Womack và Daniel T. Jones.
- *Mục tiêu: Một quá trình cải tiến liên tục*, Eliyahu M. Goldratt.
- *Triển khai phần mềm tinh gọn: Từ khái niệm đến tiền mặt*, Mary Poppendieck và Tom Poppendieck.
- *Làm cho công việc hữu hình: Vạch rõ sự thất thoát thời gian để tối ưu công việc và dòng chảy*, Dominica DeGrandis.

Chương 3

Tóm lược

Phương thức thứ hai mô tả các nguyên tắc tạo phản hồi hai chiều, nhanh và liên tục ở mọi giai đoạn của dòng giá trị. Mục tiêu là tạo ra hệ thống làm việc ngày càng an toàn và có khả năng chống chịu tốt hơn. Điều này đặc biệt quan trọng khi làm việc với hệ thống phức tạp, nơi cơ hội sớm nhất để phát hiện và sửa lỗi thường chỉ xuất hiện khi một sự cố thảm họa đã bắt đầu.

Hệ thống làm việc có thể an toàn hơn bằng việc tạo dòng thông tin nhanh, thường xuyên và chất lượng cao trong toàn bộ dòng giá trị và tổ chức, bao gồm cả vòng phản hồi ngược (*feedback*) và phản hồi xuôi (*feedforward*). Nhờ vậy, vấn đề được phát hiện và xử lý khi còn nhỏ, ít tốn kém và dễ sửa; có thể ngăn vấn đề trước khi gây thảm họa; và tạo tri thức tổ chức để đưa vào công việc tương lai. Phản hồi nhanh là yếu tố then chốt để đạt chất lượng, độ tin cậy và an toàn trong dòng giá trị công nghệ. Có thể làm điều đó bằng cách nhìn thấy vấn đề ngay khi chúng phát sinh; huy động mọi người cùng xử lý để tạo tri thức mới; đẩy chất lượng về gần nguồn phát sinh hơn; và liên tục tối ưu cho các trung tâm công việc phía sau.

Chủ đề

1. Thất bại là điều cố hữu và không thể tránh khỏi trong hệ thống phức tạp; vì vậy phải tạo hệ thống làm việc an toàn.
2. Các giả định về thiết kế và vận hành phải được kiểm nghiệm liên tục.
3. Khi vấn đề xảy ra, phải huy động bất kỳ ai cần thiết và cùng tập trung xử lý.
4. Tiếp tục đẩy chất lượng về gần nguồn phát sinh hơn, đồng thời tối ưu cho các trung tâm công việc phía sau.

Câu hỏi thảo luận

1. Những đặc điểm xác định một hệ thống phức tạp là gì?
2. Bốn điều kiện cần có để tổ chức làm việc an toàn hơn trong hệ thống phức tạp là gì?
3. Trong dòng giá trị công nghệ, tổ chức thường nhận kết quả kém vì thiếu phản hồi nhanh. Hãy nêu và thảo luận các ví dụ từ trải nghiệm cá nhân.
4. Vì sao cần huy động nhiều người cùng tập trung để khắc phục vấn đề phát sinh?
5. * Trong nghiên cứu tình huống Excella, nhóm đã tạo một hình thức vật lý và số hóa của dây Andon để làm công việc hữu hình hơn và tăng cộng tác. Có thể áp dụng cách này trong tổ chức của bạn như thế nào?

Đọc thêm

- *Tai nạn bình thường: Sống cùng các công nghệ rủi ro cao*, Charles Perrow.
- *Cẩm nang thực địa để hiểu sai sót của con người*, Tiến sĩ Sidney Dekker.
- *Nguyên lý thứ năm: Nghệ thuật và thực hành của tổ chức học hỏi*, Peter M. Senge.
- *Doanh nghiệp tinh gọn: Cách các tổ chức hiệu suất cao đổi mới ở quy mô lớn*, Jez Humble, Joanne Molesky và Barry O'Reilly.
- *Hãy khám phá!: Giảm rủi ro và tăng sự tự tin bằng kiểm thử khám phá*, Elisabeth Hendrickson.

Chú giải của người dịch - dây Andon: Trong sản xuất Toyota, người lao động có thể kéo dây Andon để báo bất thường và dừng dây chuyền nhằm xử lý lỗi ngay tại nguồn. Trong phần mềm, tương đương có thể là một cảnh báo build đỏ khiến cả nhóm ưu tiên khôi phục pipeline trước khi tiếp tục đưa thêm thay đổi vào.

Chương 4

Tóm lược

Phương thức thứ ba tập trung tạo dựng văn hóa học hỏi và thử nghiệm liên tục. Các nguyên tắc này cho phép không ngừng tạo ra tri thức cá nhân, rồi chuyển nó thành tri thức của nhóm và tổ chức. Trong dòng giá trị công nghệ, mục tiêu là xây dựng văn hóa tin cậy cao, củng cố quan niệm rằng mọi người đều là người học suốt đời và phải chấp nhận rủi ro trong công việc hằng ngày.

Bằng cách áp dụng phương pháp khoa học cho cả cải tiến quy trình lẫn phát triển sản phẩm, tổ chức học từ thành công và thất bại, nhận diện ý tưởng không hiệu quả và củng cố ý tưởng hiệu quả. Hơn nữa, mọi bài học cục bộ nhanh chóng được chuyển thành cải tiến toàn cục để toàn tổ chức có thể sử dụng kỹ thuật và thực hành mới. Cần dành riêng thời gian để cải tiến công việc hằng ngày, tiếp tục tăng tốc và bảo đảm việc học. Cũng nên liên tục đưa áp lực có kiểm soát vào hệ thống để thúc đẩy cải tiến. Tổ chức thậm chí có thể mô phỏng và chủ động đưa lỗi vào các dịch vụ production trong điều kiện kiểm soát nhằm tăng khả năng chống chịu. Bằng hệ thống học hỏi liên tục và năng động này, các nhóm có thể thích ứng nhanh, tự động với môi trường luôn thay đổi, qua đó giành lợi thế trên thị trường.

Chủ đề

1. Tạo điều kiện học hỏi trong tổ chức và xây dựng văn hóa an toàn.
2. Thể chế hóa việc cải tiến công việc hằng ngày.
3. Biến phát hiện cục bộ thành cải tiến toàn cục.
4. Đưa các mẫu hình tăng khả năng chống chịu vào công việc hằng ngày.

Câu hỏi thảo luận

1. Tổ chức có thể nuôi dưỡng văn hóa học hỏi và thử nghiệm liên tục bằng cách nào?
2. Lãnh đạo có thể đặt những câu hỏi gì để giúp nhân viên nhìn thấy và giải quyết vấn đề trong công việc hằng ngày?
3. Trong dòng giá trị công nghệ, có những cách nào đưa sức ép vào hệ thống để cải thiện cả hiệu suất và khả năng chống chịu?
4. Theo Mô hình Kiểu loại Tổ chức Westrum, bạn mô tả tổ chức của mình là bệnh hoạn (*pathological*), quan liêu (*bureaucratic*) hay kiến tạo (*generative*)?

Đọc thêm

- “Tìm lại nghiên cứu thời gian và thao tác” (*Time-and-Motion Regained*), Paul Adler, *Harvard Business Review*.
- *Phản mong manh: Những thứ được lợi từ hỗn loạn* (*Antifragile: Things That Gain from Disorder*), Nassim Nicholas Taleb.

Chú giải của người dịch - “phản mong manh”: Khái niệm *antifragile* không chỉ là “chịu được cú sốc”. Một hệ thống có khả năng chống chịu sẽ phục hồi sau sự cố; một hệ thống phản mong manh còn trở nên tốt hơn nhờ những cú sốc nhỏ, có kiểm soát. Ví dụ: diễn tập sự cố định kỳ làm lộ điểm yếu và khiến quy trình ứng cứu mạnh hơn trước khi sự cố thật xảy ra.

Chương 5

Tóm lược

Việc lựa chọn dòng giá trị để chuyển đổi DevOps cần được cân nhắc kỹ. Dòng giá trị được chọn không chỉ quyết định độ khó của chuyển đổi mà còn quyết định những ai sẽ tham gia, ảnh hưởng đến cách tổ chức nhóm và cách lãnh đạo hỗ trợ các nhóm cùng từng cá nhân hiệu quả nhất. Chương này trình bày nhiều mô hình giúp tổ chức quyết định nên bắt đầu với dòng giá trị nào.

Các dòng giá trị ứng viên được đánh giá theo nhiều chiều: dịch vụ mới hoàn toàn (*greenfield*) hay hệ thống hiện hữu (*brownfield*); hệ thống tương tác (*system of engagement*) hay hệ thống lưu trữ hồ sơ cốt lõi (*system of record*). Tổ chức cũng cần ước lượng cân bằng rủi ro/lợi ích của việc chuyển đổi và đánh giá mức phản kháng có thể đến từ các nhóm liên quan. Khi chọn nơi và cách bắt đầu một cách cẩn trọng, thành viên nhóm có thể thử nghiệm và học hỏi tại những khu vực tạo ra giá trị mà không gây nguy hiểm cho phần còn lại của tổ chức. Nhờ vậy, họ xây dựng được nền tảng ủng hộ, giành được quyền mở rộng DevOps trong tổ chức và nhận được sự ghi nhận, biết ơn từ một cộng đồng ngày càng lớn.

Chủ đề

1. Cân nhắc dịch vụ *greenfield* và *brownfield*.
2. Xem xét hệ thống lưu trữ hồ sơ cốt lõi và hệ thống tương tác.
3. Mở rộng DevOps trong toàn tổ chức.

Câu hỏi thảo luận

1. Tổ chức quyết định bắt đầu chuyển đổi DevOps ở đâu như thế nào?
2. Hãy xem xét và thảo luận cách nhóm Nordstrom khởi động sáng kiến chuyển đổi DevOps năm 2013. Họ tập trung vào những lĩnh vực nào và vì sao?
3. Một số ví dụ về dự án DevOps *greenfield* và *brownfield* là gì?
4. * Nghiên cứu tình huống chuyển đổi *brownfield* Kessel Run dạy bạn điều gì? Có thể áp dụng các bài học đó vào tổ chức hoặc nhóm của bạn không?

Đọc thêm

- *Vượt qua vực thẳm: Tiếp thị và bán sản phẩm công nghệ cao cho khách hàng đại chúng*, Geoffrey A. Moore và Regis McKenna.
- “Trí tuệ của Peter Drucker từ A đến Z”, Leigh Buchanan, *Inc.*
- “Liên tục triển khai văn hóa tại Etsy”, Michael Rembetsy và Patrick McDonnell:
<http://www.slideshare.net/mcdonnps/continuously-deploying-culture-scaling-culture-at-etsy-14588485>

Video nên xem

- “Hành trình số của Không quân trong 12 parsec hoặc ít hơn”, DevOps Enterprise Summit, London, 2020:
<https://videolibrary.doesvirtual.com/?video=467489046>
- “Cứu nền kinh tế khỏi sụp đổ (bằng một PaaS siêu quy mô)”, DevOps Enterprise Summit Europe Virtual, 2021: <https://videolibrary.doesvirtual.com/?video=550704128>

Chú giải của người dịch - *greenfield* và *brownfield*: *Greenfield* là xây mới với ít ràng buộc di sản; *brownfield* là cải tạo hệ thống đang chạy, phải giữ tương thích dữ liệu, quy trình và người dùng hiện hữu. Ví dụ, một dịch vụ thanh toán mới tách riêng có thể là *greenfield*; hiện đại hóa hệ thống lõi ngân hàng 20 năm tuổi là *brownfield*.

Chương 6

Tóm lược

Sau khi xác định một dòng giá trị để áp dụng các nguyên tắc và mẫu hình DevOps, bước tiếp theo là hiểu đủ rõ cách giá trị được chuyển giao cho khách hàng: công việc nào được thực hiện, ai thực hiện và tổ chức có thể làm gì để cải thiện dòng chảy. Chương này đi qua các bước: xác định tất cả nhóm cần thiết để tạo giá trị cho khách hàng; lập bản đồ dòng giá trị nhằm làm mọi công việc cần thiết trở nên hữu hình; và dùng bản đồ đó để hướng dẫn các nhóm tạo giá trị tốt hơn, nhanh hơn.

Tổ chức cũng phải phân bổ đủ thời gian cho cải tiến, khắc phục các vấn đề và vấn đề kiến trúc đã biết, bao gồm yêu cầu phi chức năng. Các nghiên cứu tình huống Nordstrom và LinkedIn cho thấy lead time và chất lượng có thể cải thiện mạnh mẽ khi tổ chức tìm ra vấn đề trong dòng giá trị và trả bớt nợ kỹ thuật.

Chủ đề

1. Hiểu đầy đủ cách giá trị được chuyển giao cho khách hàng.
2. Xác định các nhóm hỗ trợ dòng giá trị.
3. Lập bản đồ dòng giá trị để nhìn thấy công việc.
4. Công cụ củng cố hành vi mong muốn.

Câu hỏi thảo luận

1. Ai cần tham gia chuyển đổi DevOps và tổ chức quyết định điều đó như thế nào?
2. Bản đồ dòng giá trị mang lại hiểu biết cụ thể về cách công việc được thực hiện như thế nào, và có thể ghi lại bản đồ đó ra sao?
3. Khi một tổ chức đã có nhóm chuyển đổi chuyên trách chịu trách nhiệm cho hoạt động hằng ngày, làm thế nào để quy trách nhiệm cho nhóm về việc đạt các kết quả cấp hệ thống được xác định rõ ràng và đo lường được?

Đọc thêm

- *Mặt bên kia của đổi mới: Giải quyết thách thức thực thi*, Vijay Govindarajan và Chris Trimble.
- *Được truyền cảm hứng: Cách tạo ra sản phẩm khách hàng yêu thích*, Marty Cagan.
- “LinkedIn: Câu chuyện về nhu cầu tốc độ có lẽ không lành mạnh của Thung lũng Silicon”, Ashlee Vance, *Bloomberg*.

Lưu ý: Các câu hỏi đánh dấu * chỉ áp dụng cho Ấn bản thứ hai của *The DevOps Handbook*.

Chương 7

Tóm lược

Chương này đề nghị các nhóm bắt đầu suy nghĩ về cách tổ chức để đạt mục tiêu của dòng giá trị tốt nhất. Định luật Conway phát biểu: “Các tổ chức thiết kế hệ thống... bị ràng buộc phải tạo ra những thiết kế là bản sao của cấu trúc giao tiếp trong chính các tổ chức đó... Tổ chức càng lớn thì càng kém linh hoạt và hiện tượng này càng rõ rệt.” Nói cách khác, cách tổ chức các nhóm có ảnh hưởng mạnh mẽ đến phần mềm được tạo ra, cũng như kết quả về kiến trúc và production.

Muốn công việc chảy nhanh từ Phát triển sang Vận hành, với chất lượng cao và kết quả tốt cho khách hàng, tổ chức phải sắp xếp nhóm và công việc sao cho Định luật Conway phát huy lợi thế. Nếu thực hiện kém, Định luật Conway sẽ ngăn các nhóm làm việc an toàn, độc lập; thay vào đó, họ bị ghép nối chặt chẽ,

tất cả phải chờ nhau hoàn thành công việc, và ngay cả một thay đổi nhỏ cũng có thể gây hậu quả mang tính toàn cục, thảm họa. Các nghiên cứu tình huống Etsy và Target cho thấy kiến trúc và thiết kế tổ chức có thể cải thiện kết quả mạnh mẽ như thế nào.

Chủ đề

1. Ba kiểu cấu trúc tổ chức chính định hình cách thiết kế dòng giá trị DevOps.
2. Vấn đề thường bắt nguồn từ định hướng chức năng quá mức; cần trao quyền cho các nhóm định hướng thị trường.
3. Kiểm thử, vận hành và bảo mật là công việc của mọi người; mọi người nên có năng lực tổng quát.
4. Đừng cấp vốn cho dự án; hãy cấp vốn cho dịch vụ và sản phẩm.

Câu hỏi thảo luận

1. Các nhóm nên tổ chức đội ngũ tham gia chuyển đổi DevOps, bảo vệ năng lực làm việc và tối đa hóa cơ hội thành công như thế nào?
2. Hãy xem xét và tóm tắt hành trình Etsy trở thành một trong những tổ chức DevOps được ngưỡng mộ nhất nhờ sử dụng Sprouter.
3. Hãy định nghĩa và suy ngẫm về ba kiểu cấu trúc tổ chức chính ảnh hưởng đến cách tổ chức thiết kế dòng giá trị DevOps khi cân nhắc Định luật Conway.
4. * Hãy suy ngẫm về thiết kế nhóm trong tổ chức của bạn dựa trên bốn kiểu nhóm được nêu trong *Team Topologies*. Theo bạn, tổ chức có thể cải thiện thiết kế như thế nào?

Đọc thêm

- “Các ủy ban phát minh như thế nào?” (*How Do Committees Invent?*), Tiến sĩ Melvin E. Conway, MelConway.com.
- “Khi sự ‘thông minh’ phản tác dụng: Etsy vượt qua những lựa chọn kiến trúc kém như thế nào”, Sean Gallagher, *Ars Technica*.
- “Chuyển giao liên tục” (*Continuous Delivery*), Scott Prugh, *Scaled Agile Framework*.
- “Tải nhận thức của nhóm” (*Team Cognitive Load*), Matthew Skelton và Manuel Pais: <https://itrevolution.com/cognitive-load/>

Video nên xem

- “Nguyên khối hay microservices đã bỏ lỡ trọng tâm” (*Monoliths vs Microservices Is Missing the Point*), DevOps Enterprise Summit, Las Vegas, 2019: <https://videos.itrevolution.com/watch/504281810/>

Chú giải của người dịch - Định luật Conway: Kiến trúc phần mềm thường phản chiếu ranh giới giao tiếp của tổ chức. Ví dụ, nếu đội giao diện, API và cơ sở dữ liệu tách biệt, chỉ trao đổi qua phiếu yêu cầu, hệ thống dễ hình thành ba tầng phụ thuộc chặt. Một nhóm sản phẩm liên chức năng chịu trách nhiệm trọn vẹn có xu hướng tạo dịch vụ tự chủ hơn.

Chương 8

Tóm lược

Mục tiêu của cẩm nang này là giúp tạo ra kết quả định hướng thị trường, nơi nhiều nhóm nhỏ có thể nhanh chóng và độc lập chuyển giao giá trị cho khách hàng. Điều này có thể khó đạt được khi Vận hành được tập trung hóa và tổ chức theo chức năng. Có thể tạo kết quả định hướng thị trường tốt hơn bằng cách tích hợp năng lực Vận hành sâu hơn vào các nhóm Phát triển, giúp cả hai hiệu quả và năng suất hơn.

Chương này khảo sát nhiều cách thực hiện ở cả cấp tổ chức lẫn trong các nghi thức hằng ngày. Qua đó, Vận hành có thể cải thiện đáng kể năng suất của các nhóm Phát triển trong toàn tổ chức, đồng thời tạo điều kiện cho sự hợp tác và kết quả tổ chức tốt hơn. Tổ chức có thể dùng ba chiến lược rộng: (1) tạo năng lực tự phục vụ để lập trình viên trong các nhóm dịch vụ làm việc hiệu quả; (2) đưa kỹ sư Vận hành vào các nhóm dịch vụ; và (3) chỉ định đầu mối liên lạc Vận hành cho các nhóm dịch vụ khi không thể đưa kỹ sư Vận hành trực tiếp vào nhóm. Cuối cùng, chương mô tả cách kỹ sư Vận hành tham gia các nghi thức làm việc hằng ngày của nhóm Phát triển, gồm họp đứng hằng ngày, lập kế hoạch và họp nhìn lại.

Chủ đề

1. Tạo dịch vụ dùng chung để tăng năng suất lập trình viên.
2. Đưa kỹ sư Vận hành vào các nhóm dịch vụ.
3. Tích hợp Vận hành vào các nghi thức của Phát triển.

Câu hỏi thảo luận

1. Ba chiến lược rộng nào cho phép nhóm Vận hành tập trung đạt được kết quả thường gắn với các nhóm định hướng thị trường?
2. Vì sao và bằng cách nào tổ chức nên đưa kỹ sư Vận hành vào nhóm Phát triển và nhóm sản phẩm?
3. Hãy nêu một số ví dụ về phản hồi mà Vận hành có thể đưa vào cuộc họp nhìn lại của nhóm dự án.
4. * Hãy thảo luận lợi ích và thách thức tiềm ẩn của nhóm chức năng so với nhóm tồn tại lâu dài, đa kỹ năng như trong nghiên cứu tình huống Nationwide Building Society. Nhóm của bạn hiện được tổ chức ra sao? Việc áp dụng kiểu nhóm đa kỹ năng có đem lại lợi ích không?

Đọc thêm

- “Trò chơi phát triển sản phẩm mới” (*The New Product Development Game*), Hirotaka Takeuchi và Ikujiro Nonaka, *Harvard Business Review*.
- “DevOps Kaizen: Tìm và sửa nguyên nhân thực sự phía sau vấn đề”, Damon Edwards: <http://www.slideshare.net/dev2ops/dev-ops-kaizen-damon-edwards>
- “Disney DevOps: Tiến đến vô cực và xa hơn”, Jason Cox, DevOps Enterprise Summit 2014.

Video nên xem

- “Thực hiện sứ mệnh: Nationwide Building Society”, DevOps Enterprise Summit, London, 2020: <https://videolibrary.doesvirtual.com/?video=432109857>

Chương 9

Tóm lược

Để tạo dòng chảy nhanh và đáng tin cậy từ Phát triển sang Vận hành, tổ chức phải luôn dùng môi trường giống production ở mọi giai đoạn của dòng giá trị. Hơn nữa, các môi trường này phải được tạo tự động, lý tưởng là theo nhu cầu từ script và thông tin cấu hình lưu trong hệ thống quản lý phiên bản, đồng thời hoàn toàn tự phục vụ, không cần Vận hành thao tác thủ công.

Mục tiêu là bảo đảm có thể tái tạo toàn bộ môi trường production từ những gì được lưu trong quản lý phiên bản; đây là một trong nhiều thực hành cho thấy Vận hành có thể giúp lập trình viên năng suất hơn rất nhiều. Chương này bàn về cách xây dựng cơ chế tạo môi trường theo nhu cầu; mở rộng việc dùng quản lý phiên bản tới mọi người trong dòng giá trị; làm cho hạ tầng dễ xây dựng lại hơn sửa chữa; và bảo đảm lập trình viên chạy mã trong môi trường giống production ở mọi giai đoạn của vòng đời phát triển phần mềm.

Những thực hành này giúp giải quyết vấn đề dễ hơn, nhanh hơn, mở rộng công suất dễ hơn và tạo nền tảng cho tự động hóa kiểm thử toàn diện.

Chủ đề

1. Cho phép tạo môi trường phát triển, kiểm thử và production theo nhu cầu.
2. Tạo một kho sự thật duy nhất cho toàn hệ thống.
3. Hạ tầng nên dễ xây dựng lại hơn sửa chữa.

Câu hỏi thảo luận

1. Vì sao dùng quản lý phiên bản cho môi trường dự báo hiệu suất CNTT và tổ chức tốt hơn dùng quản lý phiên bản cho mã nguồn?
2. Đưa toàn bộ thành phần tạo nên production vào quản lý phiên bản, qua đó hình thành “nguồn sự thật duy nhất”, đem lại lợi ích gì?
3. Xây dựng lại và sửa chữa hạ tầng khác nhau thế nào? Vì sao tạo ứng dụng và môi trường dễ xây dựng lại hơn sửa chữa lại có lợi?

Đọc thêm

- *DevOps hiệu quả: Xây dựng văn hóa cộng tác, gắn kết và hệ công cụ ở quy mô lớn*, Jennifer Davis và Katherine Daniels.
- “Máy chủ của bạn là thú cưng hay gia súc?” (*Are Your Servers PETS or CATTLE?*), Simon Sharwood, *The Register*.
- “Docker và Ba Phương thức DevOps, Phần 1: Phương thức thứ nhất - Tư duy hệ thống”, John Willis, Docker.

Chú giải của người dịch - “thú cưng hay gia súc”: “Thú cưng” là máy chủ được chăm sóc riêng, hỏng thì sửa tại chỗ; “gia súc” là các máy chủ thay thế được, hỏng thì loại bỏ và tạo máy mới từ cấu hình tự động. Ấn độ nhấn mạnh khả năng tái tạo, không xem nhẹ độ tin cậy.

Chương 10

Tóm lược

Tổ chức dễ nhận kết quả không mong muốn nếu chỉ tìm và sửa lỗi trong một giai đoạn kiểm thử riêng biệt, do một bộ phận QA riêng thực hiện sau khi toàn bộ hoạt động phát triển đã hoàn tất. Nếu chỉ kiểm thử vài lần mỗi năm, lập trình viên sẽ biết về sai sót của mình sau nhiều tháng kể từ khi đưa vào thay đổi gây lỗi. Ngoài ra, nếu không có kiểm thử tự động, càng viết nhiều mã thì càng tốn nhiều thời gian và tiền bạc để kiểm thử.

Phần còn lại của chương trình bày các thực hành tích hợp liên tục cần thiết để tái tạo kết quả tốt. Chúng gồm: tạo một bộ kiểm thử tự động toàn diện để xác nhận bản build vẫn ở trạng thái đạt yêu cầu và có thể triển khai; tổ chức các bộ kiểm thử và hoạt động kiểm thử thành pipeline triển khai; và xây dựng chuẩn mực văn hóa rằng phải làm mọi điều cần thiết để đưa build trở lại trạng thái xanh nếu một thay đổi làm hỏng bất kỳ kiểm thử tự động nào. Nhờ đó, tổ chức đặt nền móng cho tích hợp liên tục, cho phép nhiều nhóm nhỏ phát triển, kiểm thử và triển khai mã vào production một cách độc lập, an toàn, đồng thời chuyển giao giá trị cho khách hàng.

Chủ đề

1. Liên tục build, kiểm thử và tích hợp mã cùng môi trường.

2. Xây dựng bộ kiểm thử xác thực tự động nhanh và đáng tin cậy.
3. Kéo dây Andon khi pipeline triển khai bị hỏng.

Câu hỏi thảo luận

1. Hãy mô tả vòng xoáy đi xuống bắt đầu từ việc tìm và sửa lỗi trong một giai đoạn kiểm thử riêng, do bộ phận QA riêng thực hiện sau khi toàn bộ phát triển đã hoàn tất.
2. Vì sao phải tạo quy trình build và kiểm thử tự động chạy trong các môi trường chuyên biệt?
3. Ba năng lực cần thiết để tạo thực hành tích hợp liên tục là gì?
4. * Hãy thảo luận ba chiến lược của Angie Jones để chuyển giao tính năng và tự động hóa kiểm thử: cộng tác, tự động hóa có chiến lược, xây dựng tăng dần. Nhóm của bạn có thể hoặc không thể áp dụng chiến lược này như thế nào?
5. * Dữ liệu từ các báo cáo *State of DevOps* cho thấy bốn thành phần thiết yếu để đạt kiểm thử tự động vượt trội là: đáng tin cậy; nhất quán; nhanh và tái lập được; bao quát. Các kiểm thử của bạn có đáp ứng tất cả không? Cần gì để đạt đủ cả bốn?

Đọc thêm

- *Chuyển giao liên tục: Phát hành phần mềm đáng tin cậy thông qua tự động hóa build, kiểm thử và triển khai*, Jez Humble và David Farley.
- “Tích hợp liên tục” (*Continuous Integration*), Martin Fowler, MartinFowler.com.
- “Hiện thực hóa cải tiến chất lượng thông qua phát triển hướng kiểm thử: Kết quả và kinh nghiệm của bốn nhóm công nghiệp”, Nachiappan Nagappan, E. Michael Maximilien, Thirumalesh Bhat và Laurie Williams, *Empirical Software Engineering*.

Video nên xem

- “Một công ty khách sạn vận hành 30 tỷ USD doanh thu trong container như thế nào”, DevOps Enterprise Summit, Las Vegas, 2020: <https://videolibrary.doesvirtual.com/?video=524020857>

Chương 11

Tóm lược

Khả năng tạo “nhánh” trong hệ thống quản lý phiên bản ban đầu chủ yếu nhằm cho phép lập trình viên làm việc song song trên các phần khác nhau của hệ thống phần mềm, không gặp rủi ro một cá nhân đưa thay đổi có thể làm mất ổn định hoặc gây lỗi vào nhánh chính. Tuy nhiên, lập trình viên càng làm việc cô lập lâu trên nhánh riêng thì việc tích hợp và hợp nhất thay đổi của mọi người trở lại nhánh chính càng khó.

Tích hợp liên tục được thiết kế để giải quyết vấn đề này bằng cách biến việc hợp nhất vào nhánh chính thành công việc hằng ngày của mọi người. Sau việc sử dụng quản lý phiên bản một cách toàn diện, tích hợp liên tục là một trong những thực hành quan trọng nhất giúp công việc chảy nhanh trong dòng giá trị, cho phép nhiều nhóm phát triển độc lập phát triển, kiểm thử và chuyển giao giá trị. Chương này mô tả những thực hành cần thiết để triển khai tích hợp liên tục và cách vượt qua các phản đối thường gặp. Phát triển dựa trên nhánh chính có lẽ là thực hành gây tranh luận nhất trong sách. Tuy vậy, dữ liệu rất rõ ràng: phát triển dựa trên nhánh chính dự báo thông lượng cao hơn, độ ổn định tốt hơn, thậm chí mức hài lòng công việc cao hơn và tỷ lệ kiệt sức thấp hơn - như các ví dụ HP LaserJet và Bazaarvoice minh họa.

Chủ đề

1. Triển khai phát triển theo lô nhỏ.

2. Điều gì xảy ra khi tổ chức không thường xuyên đưa mã vào nhánh chính.
3. Áp dụng thực hành phát triển dựa trên nhánh chính.

Câu hỏi thảo luận

1. Lý do chính tạo khả năng “phân nhánh” trong hệ thống quản lý phiên bản là gì? Ưu và nhược điểm của việc này là gì?
2. Hai cực của các chiến lược phân nhánh là gì? Hãy mô tả và nêu ví dụ.
3. Vì sao phát triển dựa trên nhánh chính gây nhiều tranh luận? Dữ liệu nào từ *2015 State of DevOps Report* của Puppet Labs cho thấy có thể triển khai thực hành này?
4. Tổ chức có thể thuyết phục lập trình viên chấp nhận phát triển dựa trên nhánh chính như thế nào?

Đọc thêm

- *Dẫn dắt chuyển đổi: Áp dụng nguyên tắc Agile và DevOps ở quy mô lớn*, Gary Gruver và Tommy Mouser.
- *Báo cáo Hiện trạng DevOps 2015*, Nigel Kersten, IT Revolution và PwC.
- “Cuộc chuyển đổi DevOps đáng kinh ngạc”, Gene Kim, ITRevolution.com.

Chú giải của người dịch - phát triển dựa trên nhánh chính: Đây không nhất thiết có nghĩa là mọi thay đổi chưa hoàn chỉnh đều lộ ra cho người dùng. Nhóm thường tích hợp thay đổi nhỏ vào nhánh chính rất thường xuyên, dùng cờ tính năng để tách việc “đã có mã” khỏi việc “mở tính năng”.

Chương 12

Tóm lược

Chương này giải thích cách giảm ma sát đi kèm việc triển khai production, bảo đảm chúng có thể được thực hiện thường xuyên và dễ dàng bởi Vận hành hoặc Phát triển. Có thể làm điều này bằng cách mở rộng pipeline triển khai. Thay vì chỉ liên tục tích hợp mã trong môi trường giống production, tổ chức nên cho phép đưa vào production bất kỳ bản build nào đã vượt qua quy trình kiểm thử và xác thực tự động, theo yêu cầu hoặc tự động.

Như các ví dụ Facebook, Etsy và CSG cho thấy, phát hành và triển khai không nhất thiết phải là những sự kiện rủi ro cao, đầy kịch tính, đòi hỏi hàng chục hoặc hàng trăm kỹ sư làm việc suốt ngày đêm. Chúng có thể trở thành công việc hoàn toàn thường lệ trong hoạt động hằng ngày của mọi người. Qua đó, tổ chức có thể giảm lead time triển khai từ hàng tháng xuống vài phút, nhanh chóng chuyển giao giá trị cho khách hàng mà không gây hỗn loạn hay gián đoạn.

Chủ đề

1. Tự động hóa quy trình triển khai.
2. Tách việc triển khai khỏi việc phát hành.
3. Khảo sát chuyển giao liên tục và triển khai liên tục trong thực tế.

Câu hỏi thảo luận

1. Facebook đã biến triển khai mã thành công việc hằng ngày của mọi người và duy trì năng suất lập trình viên như thế nào?
2. Sau khi quy trình triển khai được lập thành tài liệu, những bước thủ công nào có thể được đơn giản hóa và tự động hóa?

3. Các yêu cầu đối với một pipeline triển khai là gì?

Đọc thêm

- *Khởi nghiệp tinh gọn: Cách các doanh nhân ngày nay dùng đổi mới liên tục để tạo doanh nghiệp thành công triệt để*, Eric Ries.
- “Triển khai liên tục tại IMVU: Làm điều bất khả thi năm mươi lần một ngày”, Timothy Fitz, TimothyFitz.com.
- *Báo cáo Hiện trạng DevOps 2013*, Puppet Labs và IT Revolution Press.

Chú giải của người dịch - deployment và release: *Deployment* là đưa phiên bản phần mềm vào một môi trường; *release* là cho người dùng thực sự tiếp cận chức năng. Một nhóm có thể triển khai mã vào production nhưng giữ tính năng tắt bằng feature flag, rồi phát hành dần cho 5%, 25% và 100% người dùng.

Chương 13

Tóm lược

Kiến trúc của bất kỳ sản phẩm hay tổ chức thành công nào tất yếu sẽ tiến hóa trong suốt vòng đời. Thách thức là làm sao tổ chức có thể liên tục chuyển từ kiến trúc hiện có sang kiến trúc cần có. Các nhóm thường làm việc với kiến trúc cũ, ghép nối quá chặt; mỗi lần cố đưa mã vào nhánh chính hoặc phát hành vào production, họ có nguy cơ gây lỗi trên phạm vi toàn cục. Để tránh điều đó, mỗi thay đổi nhỏ đều đòi hỏi khối lượng phối hợp và phê duyệt khổng lồ từ mọi nhóm có thể bị ảnh hưởng. Sau đó, số thay đổi được gom vào mỗi lần triển khai tăng lên, khiến việc tích hợp và kiểm thử phức tạp hơn, đồng thời làm tăng khả năng thất bại.

Tất cả những điều này góp phần tạo nên một hệ thống làm việc không an toàn và nỗi sợ tích hợp, triển khai mã; từ đó dẫn đến vòng xoáy đi xuống tự củng cố, trong đó tổ chức ngày càng triển khai ít thường xuyên hơn. Chương này mô tả các bước để đảo ngược vòng xoáy; rà soát những kiểu mẫu kiến trúc chính; xem xét thuộc tính của kiến trúc giúp tăng năng suất lập trình viên, khả năng kiểm thử, khả năng triển khai và an toàn; và đánh giá chiến lược di chuyển an toàn từ kiến trúc hiện tại sang kiến trúc phù hợp hơn với mục tiêu tổ chức. Các nghiên cứu tình huống mô tả kỹ thuật như mẫu hình strangler, giúp nhóm chuyển đổi kiến trúc từng bước và thích ứng với nhu cầu tổ chức.

Chủ đề

1. Tạo kiến trúc hỗ trợ năng suất, khả năng kiểm thử và an toàn.
2. Rà soát các kiểu mẫu kiến trúc: nguyên khối so với microservices.
3. Dùng mẫu ứng dụng strangler để tiến hóa kiến trúc doanh nghiệp một cách an toàn.

Câu hỏi thảo luận

1. Hãy xem xét và thảo luận phương pháp eBay tái kiến trúc hướng đến hạ tầng mới cần thiết.
2. Một số hậu quả của kiến trúc ghép nối quá chặt là gì?
3. Hãy so sánh và đối chiếu hai kiểu mẫu kiến trúc nguyên khối và microservices.

Đọc thêm

- “Chuyển giao liên tục là gì?” (*What Is Continuous Delivery?*), Jez Humble, ContinuousDelivery.com.
- *Kiến trúc và mẫu hình cho Quản lý Dịch vụ CNTT, hoạch định nguồn lực và quản trị: Đóng giày cho con của người thợ giày*, Charles Betz.

- “Từ nguyên khối đến microservices”, Randy Shoup:

<http://www.slideshare.net/RandyShoup/goto-aarhus2014-enterprisearchitecturemicroservices>

Chú giải của người dịch - mẫu strangler: Thay vì viết lại toàn bộ hệ thống trong một lần, ta đặt lớp giao tiếp quanh hệ thống cũ rồi dần chuyển từng chức năng sang thành phần mới. Ví dụ, yêu cầu “tra cứu đơn hàng” được định tuyến sang dịch vụ mới, trong khi “đối soát” vẫn do hệ thống cũ xử lý cho tới lượt chuyển tiếp theo.

Chương 14

Tóm lược

Nghiên cứu cho thấy các tổ chức có hiệu suất tốt nhất rất giỏi chẩn đoán và khắc phục sự cố dịch vụ. Những tổ chức hiệu suất cao dùng phương pháp giải quyết vấn đề có kỷ luật, dựa vào telemetry production để hiểu các yếu tố có thể góp phần gây sự cố và tập trung nỗ lực xử lý.

Để tạo điều kiện cho cách giải quyết vấn đề có kỷ luật này, tổ chức cần thiết kế hệ thống sao cho nó liên tục tạo telemetry. Theo nghĩa rộng, telemetry là “một quy trình truyền thông tự động, trong đó phép đo và dữ liệu khác được thu thập tại các điểm ở xa, sau đó truyền đến thiết bị tiếp nhận để giám sát”. Telemetry nên được tạo trong ứng dụng và môi trường, ở cả production lẫn tiền production, cũng như trong pipeline triển khai. Tổ chức phải luôn có đủ telemetry để xác nhận dịch vụ đang hoạt động đúng trên production.

Khi vấn đề xảy ra, telemetry giúp nhanh chóng xác định điều gì đang sai và đưa ra quyết định có căn cứ về cách khắc phục tốt nhất, lý tưởng là từ rất lâu trước khi khách hàng bị ảnh hưởng. Kết quả không chỉ là khách hàng hài lòng hơn; việc giảm chữa cháy và khủng hoảng khi sự cố xảy ra còn tạo nơi làm việc vui vẻ, năng suất hơn, ít căng thẳng và kiệt sức hơn.

Chủ đề

1. Tạo hạ tầng telemetry tập trung.
2. Tạo telemetry nhật ký ứng dụng hữu ích cho production.
3. Dùng telemetry để định hướng giải quyết vấn đề.
4. Cho phép tạo chỉ số production như một phần của công việc hằng ngày.
5. Tìm và lấp đầy mọi khoảng trống telemetry.

Câu hỏi thảo luận

1. Hãy định nghĩa “telemetry”, thảo luận cách dùng và lợi ích của nó.
2. Tổ chức có thể thiết kế, phát triển ứng dụng và môi trường như thế nào để chúng tạo đủ telemetry?
3. Telemetry cho phép tổ chức dùng phương pháp khoa học để xây dựng giả thuyết về nguyên nhân của một vấn đề và những gì cần làm để giải quyết. Hãy nêu ví dụ về các câu hỏi cần trả lời trong quá trình xử lý kiểu này.

Đọc thêm

- *Cẩm nang Visible Ops: Triển khai ITIL trong 4 bước thực tế và có thể kiểm toán*, Gene Kim, Kevin Behr và George Spafford.
- “Đo mọi thứ, đo bất cứ thứ gì” (*Measure Anything, Measure Everything*), Ian Malpass, CodeAsCraft.com.
- *Nghệ thuật giám sát (The Art of Monitoring)*, James Turnbull.

Chú giải của người dịch - telemetry: Telemetry không chỉ là log. Nó thường gồm chỉ số định lượng (CPU, độ trễ, tỷ lệ lỗi), log sự kiện và trace theo dõi một yêu cầu qua nhiều dịch vụ. Ví dụ, cảnh báo “tỷ lệ lỗi thanh toán

tăng từ 0,2% lên 3% sau bản triển khai 14:05” hữu ích hơn một dòng log rời rạc không có thời điểm và phiên bản.

Chương 15

Tóm lược

Chương này xây dựng các công cụ cho phép tổ chức phát hiện sai lệch và những tín hiệu hỏng hóc ngày càng yếu ẩn trong telemetry production, nhờ đó ngăn ngừa sự cố thảm họa. Chương trình bày nhiều kỹ thuật thống kê và trực quan hóa - bao gồm phát hiện giá trị ngoại lai - để phân tích telemetry và dự đoán vấn đề tốt hơn, kèm các nghiên cứu tình huống minh họa cách sử dụng.

Một nghiên cứu tình huống từ Netflix làm nổi bật cách rất cụ thể để dùng telemetry giảm nhẹ vấn đề trước khi ảnh hưởng đến khách hàng. Netflix dùng các kỹ thuật này để chủ động loại máy chủ tính toán khỏi production và tự động co giãn hạ tầng tính toán. Chương cũng bàn về cách dùng trung bình trượt và bộ lọc Kolmogorov-Smirnov, hai kỹ thuật có trong nhiều công cụ vẽ biểu đồ telemetry phổ biến. Các kỹ thuật phân tích telemetry này giúp tổ chức giải quyết vấn đề nhanh hơn, rẻ hơn và sớm hơn bao giờ hết, trước khi khách hàng hay bất kỳ ai trong tổ chức bị ảnh hưởng; đồng thời cung cấp thêm ngữ cảnh cho dữ liệu, hỗ trợ quyết định tốt hơn và đạt mục tiêu tổ chức.

Chủ đề

1. Dùng giá trị trung bình và độ lệch chuẩn để phát hiện vấn đề tiềm ẩn.
2. Gắn đo lường và cảnh báo cho các kết quả không mong muốn.
3. Sử dụng kỹ thuật phát hiện bất thường.

Câu hỏi thảo luận

1. Có thể dùng nhiều kỹ thuật thống kê khác nhau để phân tích telemetry production, nhằm tìm và sửa vấn đề khi chúng còn nhỏ, trước khi gây hậu quả thảm họa. Hãy mô tả và cân nhắc cách đưa một số kỹ thuật vào thực tế.
2. Hãy xem xét và tóm tắt cách Netflix phân tích telemetry để chủ động tìm, khắc phục vấn đề trước khi khách hàng bị ảnh hưởng.
3. Nếu máy chủ web NGINX của một tổ chức ngừng phản hồi yêu cầu, những chỉ báo sớm nào có thể cảnh báo trước rằng nó đang bắt đầu lệch khỏi trạng thái vận hành bình thường?

Đọc thêm

- “Khảo sát các phương pháp phát hiện giá trị ngoại lai”, Victoria Hodge và Jim Austin, *Artificial Intelligence Review*.
- “Đừng chỉ giám sát xem dịch vụ của bạn có đang hoạt động hay không!”, Tom Limoncelli, *EverythingSysAdmin.com*.
- “Phát hiện bất thường: Một khảo sát”, Varun Chandola, Arindam Banerjee và Vipin Kumar, *ACM Computing Surveys*.

Chú giải của người dịch - trung bình trượt và bộ lọc K-S: Trung bình trượt làm mượt nhiễu để lộ xu hướng; chẳng hạn, độ trễ trung bình theo cửa sổ 5 phút tăng đều có thể báo trước bão hòa. Kiểm định Kolmogorov-Smirnov so sánh hai phân phối; nó có thể phát hiện rằng hình dạng phân phối độ trễ hôm nay đã khác đáng kể so với đường cơ sở, ngay cả khi giá trị trung bình chưa thay đổi nhiều.

Chương 16

Tóm lược

Cần tạo các cơ chế phản hồi cho phép cải thiện sức khỏe dòng giá trị ở mọi giai đoạn trong vòng đời dịch vụ, từ thiết kế sản phẩm, phát triển, triển khai, vận hành cho đến khi ngừng sử dụng. Nhờ đó, tổ chức bảo đảm dịch vụ “sẵn sàng cho production” ngay từ giai đoạn sớm nhất của dự án, đồng thời tích hợp bài học từ mỗi lần phát hành và mỗi sự cố production vào công việc tương lai, giúp mọi người an toàn và năng suất hơn.

Một số cơ chế phản hồi gồm: triển khai thay đổi vào production; sửa mã khi có sự cố và kỹ sư nhận cảnh báo trực; để lập trình viên theo sát công việc của mình đến các khâu phía sau; tạo yêu cầu phi chức năng giúp nhóm Phát triển viết mã sẵn sàng cho production hơn; hoặc thậm chí bàn giao lại dịch vụ có vấn đề để Phát triển tự quản lý. Bằng việc tạo các vòng phản hồi này, tổ chức có thể làm cho triển khai production an toàn hơn, tăng mức sẵn sàng production của mã do Phát triển tạo ra và giúp Phát triển cùng Vận hành có quan hệ làm việc tốt hơn nhờ củng cố mục tiêu, trách nhiệm và sự đồng cảm chung.

Chủ đề

1. Dùng telemetry để làm cho triển khai an toàn hơn.
2. Để lập trình viên theo sát công việc đến các khâu phía sau.
3. Ban đầu, để lập trình viên tự quản lý dịch vụ production của mình.

Câu hỏi thảo luận

1. Bí quyết của dòng chảy trơn tru và liên tục là gì?
2. Hãy mô tả các cơ chế phản hồi giúp tổ chức cải thiện sức khỏe của dòng giá trị ở mọi giai đoạn trong vòng đời dịch vụ.
3. Khi tạo các yêu cầu ra mắt mà dịch vụ phải đáp ứng trước khi tương tác với khách hàng thật và tiếp nhận lưu lượng production thật, tổ chức giúp mọi nhóm sản phẩm hưởng lợi từ kinh nghiệm tích lũy, tập thể của toàn tổ chức, đặc biệt là Vận hành. Một số ví dụ về hướng dẫn và yêu cầu ra mắt là gì?
4. Cần đặt những câu hỏi nào để xác định một dịch vụ có chịu các mục tiêu tuân thủ pháp lý nào hay có khả năng phải tuân thủ trong tương lai?
5. Hãy mô tả cơ chế bàn giao trả lại dịch vụ và nêu chi tiết mục đích của nó.

Đọc thêm

- “Triển khai liên tục - Tính năng bảo mật số 1 mới, từ BSidesLA 2012”, Nick Galbreath:
<http://www.slideshare.net/nickgsuperstar/continuous-deployment-the-new-1-security-feature>
- “Các sai lầm thường gặp trong Vận hành”, Arup Chakrabarti:
<http://www.heavybit.com/library/video/common-ops-mistakes/>

Chương 17

Tóm lược

Trước khi xây dựng một tính năng, tổ chức nên tự hỏi nghiêm túc: “Chúng ta có nên xây dựng nó không, và vì sao?” Sau đó, họ nên thực hiện những thử nghiệm nhanh nhất, ít tốn kém nhất để dùng nghiên cứu người dùng xác thực xem tính năng dự kiến có thực sự tạo ra kết quả mong muốn hay không. Có thể dùng các kỹ thuật như phát triển dựa trên giả thuyết, phiếu thu hút khách hàng và kiểm thử A/B.

Những kỹ thuật này cho phép tổ chức thử nghiệm với người dùng một cách an toàn, dễ dàng; giải phóng sức sáng tạo và đổi mới; và tạo học hỏi trong tổ chức. Tổ chức càng có thể thử nghiệm, lặp lại và tích hợp phản hồi vào sản phẩm hoặc dịch vụ nhanh thì càng học nhanh và thử nghiệm vượt đối thủ. Tốc độ tích hợp phản hồi phụ thuộc vào khả năng triển khai và phát hành phần mềm. Dù thành công là quan trọng, tri thức tổ chức hình thành từ thử nghiệm cũng giúp nhân viên có tinh thần làm chủ đối với mục tiêu kinh doanh và sự hài lòng của khách hàng.

Chủ đề

1. Lịch sử kiểm thử A/B.
2. Tích hợp kiểm thử A/B vào kiểm thử tính năng, phát hành và lập kế hoạch.

Câu hỏi thảo luận

1. Hãy phân tích chính xác cách Intuit, Inc. dùng phát triển dựa trên giả thuyết, phễu thu hút khách hàng và kiểm thử A/B để tạo sản phẩm tốt, thúc đẩy học hỏi trong tổ chức và giành lợi thế trên thị trường.
2. Tổ chức có thể làm gì để kiểm thử A/B nhanh và lặp lại nhiều vòng?
3. Hãy nêu một ví dụ về giả thuyết trong phát triển tính năng theo khuôn khổ của Barry O'Reilly.

Đọc thêm

- “Lãnh đạo trong thời đại Agile: Phỏng vấn Scott Cook”, Scott Cook, Intuit.com.
- “Cách triển khai phát triển dựa trên giả thuyết”, Barry O'Reilly, BarryOReilly.com.
- “Học hỏi trong tổ chức và năng lực cạnh tranh: Nhìn lại câu chuyện ‘Allspaw/Hammond triển khai 10 lần mỗi ngày tại Flickr’”, Gene Kim, ITRevolution.com.

Chú giải của người dịch - giả thuyết sản phẩm: Một giả thuyết tốt phải kiểm chứng được, ví dụ: “Nếu rút biểu mẫu đăng ký từ 8 trường xuống 4, tỷ lệ hoàn tất của người dùng mới sẽ tăng ít nhất 10% trong hai tuần mà không làm tăng gian lận.” Nó nêu thay đổi, đối tượng, kết quả và điều kiện đo lường.

Chương 18

Tóm lược

Mục tiêu của chương này là giúp Phát triển và Vận hành giảm rủi ro của thay đổi production trước khi thực hiện. Theo truyền thống, khi rà soát thay đổi để triển khai, tổ chức thường phụ thuộc nặng vào các bước xem xét, kiểm tra và phê duyệt ngay trước giờ triển khai. Nhiều khi, phê duyệt đến từ nhóm bên ngoài, quá xa công việc để có thể quyết định sáng suốt một thay đổi có rủi ro hay không; thời gian xin đủ phê duyệt cũng kéo dài lead time của thay đổi.

Chương này tích hợp các thực hành để chuyển trọng tâm khỏi kiểm tra, phê duyệt định kỳ sang rà soát đồng cấp được tích hợp liên tục trong công việc hằng ngày. Phát triển, Vận hành và An toàn Thông tin nên cộng tác liên tục để các thay đổi đối với hệ thống vận hành đáng tin cậy, bảo mật, an toàn và đúng như thiết kế. Điều này giảm rủi ro kết quả triển khai kém, qua đó giảm phụ thuộc vào quy trình phê duyệt. Các nghiên cứu tình huống GitHub và Target cho thấy những thực hành này không chỉ cải thiện kết quả mà còn giảm đáng kể lead time và tăng năng suất lập trình viên.

Chủ đề

1. Hiểu mối nguy của quy trình phê duyệt thay đổi.
2. Tạo điều kiện phối hợp và lên lịch thay đổi.
3. Cho phép rà soát đồng cấp đối với thay đổi và lập trình cặp.

4. Mạnh dạn cắt giảm quy trình quan liêu.

Câu hỏi thảo luận

1. Quy trình pull request là gì, và GitHub dùng nó trong rà soát đồng cấp như thế nào để tăng chất lượng, làm cho triển khai an toàn và tích hợp vào dòng công việc hằng ngày?
2. Hãy nêu ví dụ về mối nguy của quy trình phê duyệt thay đổi và việc kiểm soát thay đổi quá mức.
3. Kiến trúc tương quan thế nào với nhu cầu giao tiếp, phối hợp cùng các nhóm phụ trách thành phần khác?
4. * Hãy thảo luận nghiên cứu tình huống Adidas về rà soát thay đổi tự động. Quy trình phê duyệt thay đổi trong tổ chức của bạn là gì, và bạn có thể áp dụng các nguyên tắc trong sách để cải thiện nó như thế nào?

Đọc thêm

- “Triển khai tại GitHub”, Jake Douglas, GitHub.com.
- “Công việc tri thức tinh gọn”, Bradley Staats và David M. Upton, *Harvard Business Review*.
- “Lập trình cặp so với rà soát mã”, Jeff Atwood, CodingHorror.com.
- *Nghệ thuật tinh tế của bộ máy quan liêu: Chuyển đổi số với con khỉ, dao cạo và đô vật sumo*, Mark Schwartz.

Chương 19

Tóm lược

Để làm việc an toàn trong các hệ thống phức tạp, tổ chức phải chẩn đoán và tự cải tiến tốt hơn; thành thạo việc phát hiện, giải quyết vấn đề và nhân rộng tác động bằng cách cung cấp giải pháp cho toàn tổ chức. Điều này tạo ra một hệ thống học hỏi năng động, cho phép tổ chức hiểu sai lầm và chuyển hiểu biết đó thành hành động ngăn sai lầm tái diễn. Kết quả là các tổ chức có khả năng chống chịu, giải phát hiện và giải quyết vấn đề, đồng thời nhân rộng hiệu quả bằng cách chia sẻ giải pháp trong toàn tổ chức.

Chương này khám phá cách tạo hệ thống học hỏi, thiết lập văn hóa công bằng, cũng như thường xuyên diễn tập và chủ ý tạo lỗi để tăng tốc học hỏi. Khi được xử lý đúng, sai sót vốn có trong hệ thống phức tạp có thể tạo môi trường học hỏi năng động, nơi mọi bên liên quan cảm thấy đủ an toàn để nêu ý tưởng và quan sát; các nhóm cũng phục hồi dễ dàng hơn sau những dự án không đạt kỳ vọng. Cả buổi kiểm không đổ lỗi và việc chủ động đưa lỗi vào production đều củng cố một nền văn hóa trong đó mọi người vừa cảm thấy thoải mái, vừa có trách nhiệm đưa thất bại ra ánh sáng và học từ chúng.

Chủ đề

1. Thiết lập văn hóa công bằng và học hỏi.
2. Lên lịch hậu kiểm không đổ lỗi sau sự cố, rồi công bố rộng rãi.
3. Định nghĩa lại thất bại và khuyến khích chấp nhận rủi ro có tính toán.
4. Chủ động đưa lỗi vào production để tăng khả năng chống chịu và học hỏi.

Câu hỏi thảo luận

1. Dấu hiệu đặc trưng của một tổ chức có khả năng chống chịu là gì? Hãy nêu ví dụ.
2. Hãy suy ngẫm về khái niệm “văn hóa công bằng” và đối chiếu với trải nghiệm cá nhân trong các văn hóa làm việc khác.

3. Điều gì diễn ra trong một buổi hậu kiểm không đổ lỗi?

Đọc thêm

- *Văn hóa công bằng: Cân bằng an toàn và trách nhiệm giải trình*, Tiến sĩ Sidney Dekker.
- “Hậu kiểm không đổ lỗi và văn hóa công bằng”, John Allspaw, CodeAsCraft.com.
- “Chiến lược học từ thất bại”, Amy C. Edmondson, *Harvard Business Review*.
- *Hãy phát hành!:* Thiết kế và triển khai phần mềm sẵn sàng cho production, Michael T. Nygard.

Video nên xem

- “Chúng tôi biến sự cố gián đoạn tồi tệ nhất của công ty thành một cơ hội học hỏi mạnh mẽ như thế nào”, DevOps Enterprise Summit, London, 2020: <https://videolibrary.doesvirtual.com/?video=431872263>

Chú giải của người dịch - không đổ lỗi không có nghĩa là không chịu trách nhiệm: Hậu kiểm không đổ lỗi tránh kết luận đơn giản rằng “một người đã bắt cần”. Thay vào đó, nó hỏi vì sao hành động ấy có vẻ hợp lý với thông tin khi đó, kiểm soát nào thiếu hoặc tín hiệu nào mơ hồ, rồi giao trách nhiệm rõ ràng cho các cải tiến hệ thống.

Chương 20

Tóm lược

Tổ chức nên tạo cơ chế để kiến thức và cải tiến mới được phát hiện cục bộ có thể được ghi lại, chia sẻ trên toàn tổ chức, nhờ đó nhân rộng tác động của tri thức và cải tiến toàn cục. Các kỹ thuật trong chương này cho phép đưa mỗi bài học mới vào kho tri thức tập thể của tổ chức và khuếch đại ảnh hưởng của nó.

Điều này được thực hiện bằng cách chủ động truyền đạt tri thức mới trên diện rộng, chẳng hạn qua phòng chat và thông qua công nghệ như kiến trúc dưới dạng mã, kho mã nguồn dùng chung và tiêu chuẩn hóa công nghệ. Nhờ đó, các nhóm nâng tầm thực hành không chỉ của Phát triển và Vận hành mà của cả tổ chức, để bất kỳ ai thực hiện công việc cũng làm dựa trên kinh nghiệm tích lũy của toàn tổ chức.

Chủ đề

1. Tự động hóa và ghi lại tri thức tổ chức.
2. Tạo một kho mã nguồn duy nhất dùng chung.
3. Bảo đảm lựa chọn công nghệ hỗ trợ đạt mục tiêu tổ chức.
4. Chủ động truyền đạt tri thức mới trên diện rộng để kinh nghiệm tích lũy không ngừng tăng lên.

Câu hỏi thảo luận

1. Việc để tự động hóa thực hiện công việc ngay trong phòng chat có những lợi ích gì so với chạy script tự động bằng dòng lệnh?
2. Hãy mô tả một số kết quả giá trị từ kho mã nguồn dùng chung khổng lồ trên toàn tổ chức của Google.
3. Việc triển khai yêu cầu phi chức năng đem lại điều gì cho dịch vụ của tổ chức? Một số ví dụ về yêu cầu phi chức năng là gì?
4. * Tổ chức của bạn có ở trên đám mây không? Nếu có, hãy thảo luận hạ tầng đám mây. Nó có tuân theo năm đặc tính mô tả ở trang 330-331 không? Nếu chưa, bạn có thể làm gì để đạt được?

Đọc thêm

- “Quy trình phần mềm cũng là phần mềm” (*Software Processes Are Software Too*), Leon Osterweil: <http://www.cs.unibo.it/cianca/wwwpages/ids/lettture/Osterweil.pdf>

- “ChatOps tại GitHub”, Jesse Newland: <https://speakerdeck.com/jnewland/chatops-at-github>
- “Google có 2 tỷ dòng mã - và tất cả đều ở một nơi”, Cade Metz, *Wired*.

Video nên xem

- “Quản trị công nghệ bằng sức mạnh đám đông”, DevOps Enterprise Summit, San Francisco, 2018: <https://videolibrary.doesvirtual.com/?video=524020857>

Chú giải của người dịch - ChatOps: ChatOps đưa thao tác vận hành và kết quả của chúng vào kênh trò chuyện chung. Ví dụ, lệnh triển khai được bot thực hiện trong phòng chat; mọi người cùng thấy ai yêu cầu, phiên bản nào được triển khai, kiểm tra nào đã chạy và kết quả ra sao. Kênh chat vừa là giao diện cộng tác vừa tạo dấu vết kiểm toán.

Chương 21

Tóm lược

Một thực hành thuộc Hệ thống Sản xuất Toyota được gọi là “đợt cải tiến tập trung” (*improvement blitz*): một khoảng thời gian chuyên biệt, tập trung để xử lý một vấn đề cụ thể, thường kéo dài vài ngày. Kết quả của nhóm thực hiện có thể là một cách tiếp cận mới để giải quyết vấn đề. Họ cũng có thể để lại danh sách những thay đổi cần thực hiện về sau.

Chương này mô tả các phương pháp dành thời gian cho học hỏi và cải tiến tổ chức, tiếp tục thể chế hóa thực hành dành thời gian cải thiện công việc hằng ngày. Những nghi thức này củng cố văn hóa rằng mọi người đều là người học suốt đời và việc cải tiến công việc hằng ngày được coi trọng hơn chính công việc thường nhật. Có thể thực hiện bằng cách dành thời gian trả nợ kỹ thuật và tạo diễn đàn để mọi người học hỏi, giảng dạy lẫn nhau cả trong lẫn ngoài tổ chức. Chuyên gia cũng nên sẵn sàng hỗ trợ nhóm nội bộ qua huấn luyện hoặc tư vấn. Khi mọi người giúp nhau học trong công việc hằng ngày, tổ chức có thể học nhanh hơn đối thủ và giành lợi thế trên thị trường.

Chủ đề

1. Thể chế hóa các nghi thức để trả nợ kỹ thuật.
2. Cho phép mọi người vừa dạy vừa học.
3. Tạo năng lực tư vấn và huấn luyện nội bộ để lan tỏa thực hành.

Câu hỏi thảo luận

1. Hãy mô tả một đợt cải tiến tập trung, mục tiêu và kết quả của nó.
2. Tổ chức có thể làm gì để tạo văn hóa học hỏi năng động, nơi mọi người không chỉ được học mà còn được dạy?
3. Vì sao tổ chức nên khuyến khích kỹ sư dự hội nghị, thuyết trình và khi cần thì tự tạo, tổ chức hội nghị nội bộ hoặc bên ngoài? Hãy nêu một số ví dụ từ trải nghiệm cá nhân tại hội nghị.
4. * Hoạt động học hỏi có bị mắc kẹt trong các “bong bóng” ở tổ chức của bạn không? Bạn đã thử phá vỡ những bong bóng đó như thế nào? Hãy thảo luận vòng lặp học hỏi ASREDS và khả năng áp dụng trong tổ chức.

Đọc thêm

- “Fixits, hay Tôi là Hải mã” (*Fixits, or I Am the Walrus*), Mike Bland, Mike-Bland.com.
- “Tuần đảo ngược hàng đợi” (*Queue Inversion Week*), Hal Pomeranz, Righteous IT.
- “Ngày hack: Không chỉ dành cho người Facebook”, Jessica Stillman, *Inc*.

- *Sớm hơn, an toàn hơn, hạnh phúc hơn: Phản mẫu và mẫu hình cho sự linh hoạt kinh doanh*, Jonathan Smart.

Chú giải của người dịch - improvement blitz: Có thể xem đây là một “đợt Kaizen” ngắn, có phạm vi rõ. Ví dụ, trong ba ngày, một nhóm liên chức năng chỉ tập trung giảm thời gian pipeline từ 45 xuống 15 phút; họ đo đường cơ sở, loại bước chờ, song song hóa kiểm thử và ghi lại phần việc tiếp theo.

Chương 22

Tóm lược

DevOps có thể là một trong những cách tốt nhất để tích hợp An toàn Thông tin vào công việc hằng ngày của mọi người trong dòng giá trị công nghệ. Khi An toàn Thông tin được tổ chức thành một silo nằm ngoài Phát triển và Vận hành, nhiều vấn đề phát sinh. Nếu không có tự động hóa và không tích hợp An toàn Thông tin vào công việc hằng ngày của Dev và Ops, bộ phận này chỉ có thể kiểm tra tuân thủ - điều trái ngược với kỹ nghệ bảo mật.

Chương này mô tả cách tích hợp mục tiêu An toàn Thông tin vào công việc hằng ngày để tổ chức nâng cao năng suất, an toàn và bảo mật cho cả Phát triển lẫn Vận hành. Có thể làm bằng cách đưa các kiểm soát bảo mật vào những cơ chế tổ chức đã tạo ra; bảo đảm mọi môi trường được cấp theo nhu cầu cũng ở trạng thái gia cố và giảm rủi ro; tích hợp kiểm thử bảo mật vào pipeline triển khai; và bảo đảm tạo telemetry bảo mật trong môi trường tiền production lẫn production. Nhờ đó, năng suất Phát triển và Vận hành có thể tăng đồng thời với mức an toàn tổng thể.

Chủ đề

1. Tích hợp bảo mật vào demo cuối vòng lặp phát triển, theo dõi lỗi, hậu kiểm và pipeline triển khai.
2. Bảo đảm an toàn cho ứng dụng, chuỗi cung ứng phần mềm và môi trường.
3. Tạo telemetry bảo mật trong ứng dụng và môi trường.

Câu hỏi thảo luận

1. Vì sao phải tích hợp mục tiêu An toàn Thông tin vào công việc hằng ngày?
2. Để cung cấp thư viện hoặc dịch vụ bảo mật mà mọi ứng dụng, môi trường hiện đại cần có, đồng thời cung cấp cho Dev và Ops cấu hình bảo mật hiệu quả cho các thành phần trong ngăn xếp ứng dụng, có thể đưa những mục nào vào kho mã nguồn và dịch vụ dùng chung?
3. Những loại phân tích, quét và ký số nào nên được đưa vào kiểm thử đơn vị hoặc kiểm thử chức năng tự động để có thể chạy liên tục trong pipeline triển khai?
4. Hãy liệt kê một số kiểm soát giám sát giúp bảo đảm mọi instance production khớp với trạng thái tốt đã biết.
5. * Hãy thảo luận phân tích báo cáo DBIR ở trang 364-368. Tổ chức của bạn có các thuộc tính của tổ chức hiệu suất cao không? Bạn có thể làm gì để đạt được?

Đọc thêm

- “Tấn công pipeline - Khi bảo mật gặp chuyển giao liên tục”, James Wickett:
<http://www.slideshare.net/wickett/attacking-pipeline-security-meets-continuous-delivery>
- “DevOpsSec: Áp dụng nguyên tắc DevOps vào bảo mật, DevOpsDays Austin 2012”, Nick Galbreath:
<http://www.slideshare.net/nickgsuperstar/devopssec-apply-devops-principles-to-security>
- “Xây tuân thủ từng viên gạch: Xây dựng nền tảng quản lý rủi ro”, Mossadeq Zia, Gabriel Ramirez và Noah Kunin, 18F.

Video nên xem

- “Fannie Mae dùng sự linh hoạt để hỗ trợ chủ nhà và người thuê nhà như thế nào”, DevOps Enterprise Summit, Las Vegas, 2020: <https://videolibrary.doesvirtual.com/?video=467488997>

Chương 23

Tóm lược

Chương này xem xét cách bảo vệ pipeline triển khai, cũng như cách đạt mục tiêu bảo mật và tuân thủ trong môi trường kiểm soát, bao gồm quản lý thay đổi và phân tách nhiệm vụ. Hầu như mọi tổ chức CNTT có quy mô đáng kể đều có sẵn quy trình quản lý thay đổi - các kiểm soát chính để giảm rủi ro vận hành và bảo mật. Người quản lý tuân thủ và quản lý bảo mật dựa vào quy trình này để đáp ứng yêu cầu tuân thủ; họ thường cần bằng chứng rằng mọi thay đổi đã được phê duyệt phù hợp.

Nếu pipeline triển khai được xây dựng đúng để các lần triển khai có rủi ro thấp, phần lớn thay đổi không cần qua quy trình phê duyệt thủ công, vì tổ chức đã đặt niềm tin vào các kiểm soát như kiểm thử tự động và giám sát production chủ động. Chương này bàn về các thực hành biến An toàn Thông tin thành công việc của mọi người, nơi tất cả mục tiêu An toàn Thông tin được tích hợp vào công việc hằng ngày của mọi người trong dòng giá trị. Nhờ đó, tổ chức cải thiện đáng kể hiệu quả kiểm soát để phòng ngừa vi phạm bảo mật tốt hơn, đồng thời phát hiện và phục hồi nhanh hơn. Cách làm này cũng giảm mạnh công sức chuẩn bị và vượt qua kiểm toán tuân thủ.

Chủ đề

1. Tích hợp bảo mật và tuân thủ vào quy trình phê duyệt thay đổi.
2. Phân loại lại phần lớn thay đổi ít rủi ro thành thay đổi tiêu chuẩn.
3. Giảm phụ thuộc vào phân tách nhiệm vụ.
4. Bảo đảm có tài liệu và bằng chứng cho kiểm toán viên, cán bộ tuân thủ.

Câu hỏi thảo luận

1. Ba loại quy trình thay đổi theo ITIL là gì? Hãy nêu ví dụ cho từng loại.
2. Hãy mô tả kiểm thử phá hủy và nêu ví dụ về trường hợp sử dụng có lợi.
3. Phân tách nhiệm vụ có thể kém hiệu quả hoặc cản trở tiến độ như thế nào, và có thể dùng kiểm soát nào thay thế?

Đọc thêm

- *Bộ công cụ phòng vệ kiểm toán DevOps*, James DeLuccia, Jeff Gallimore, Gene Kim và Byron Miller.
- *Chuyển tiếp Dịch vụ ITIL (ITIL Service Transition)*, AXELOS.
- “Agile 2013: Thực hiện chuyển đổi DevOps tại Salesforce”, Dave Mangot và Karthik Rajan: <http://www.slideshare.net/dmangot/agile2013effectingadevopstransformation-atsalesforce>

Chú giải của người dịch - phân tách nhiệm vụ: Mục tiêu là tránh một người có toàn quyền vừa tạo vừa phê duyệt thay đổi. Trong pipeline tự động, mục tiêu kiểm soát vẫn có thể đạt bằng rà soát đồng cấp, quyền truy cập theo vai trò, kiểm thử bắt buộc, log bất biến và phê duyệt dựa trên rủi ro - thay vì buộc mọi thay đổi nhỏ phải chờ một hội đồng hợp thủ công.

Kết luận

Tóm lược

Trong thời đại mọi lãnh đạo công nghệ đều phải đáp ứng đồng thời bảo mật, độ tin cậy và sự linh hoạt; khi vi phạm bảo mật, áp lực đưa sản phẩm ra thị trường và những cuộc chuyển đổi công nghệ quy mô lớn đang diễn ra, DevOps mang đến một giải pháp. Cẩm nang này đã giúp người đọc hiểu sâu vấn đề và cung cấp lộ trình tạo ra các giải pháp phù hợp.

Nếu không được quản lý, xung đột cố hữu giữa Phát triển và Vận hành có thể tạo ra vấn đề ngày càng tồi tệ: sản phẩm và tính năng mới ra thị trường chậm hơn; chất lượng kém; sự cố gián đoạn và nợ kỹ thuật tăng; năng suất kỹ thuật giảm; nhân viên bất mãn và kiệt sức hơn. Tuy nhiên, nguyên tắc và mẫu hình DevOps cho phép tổ chức phá vỡ xung đột cốt lõi, dai dẳng này.

Chuyển đổi DevOps có thể tạo nên tổ chức học hỏi năng động, đạt kết quả xuất sắc về dòng chảy nhanh, độ tin cậy và bảo mật đẳng cấp thế giới, đồng thời tăng năng lực cạnh tranh và sự hài lòng của nhân viên. DevOps có thể đòi hỏi chuẩn mực văn hóa và quản trị mới, cũng như thay đổi thực hành kỹ thuật và kiến trúc. Điều đó cần một liên minh xuyên suốt lãnh đạo kinh doanh, Quản lý Sản phẩm, Phát triển, QA, Vận hành CNTT, An toàn Thông tin và cả Tiếp thị - nơi nhiều sáng kiến công nghệ khởi nguồn.

Khi tất cả nhóm này cùng làm việc, tổ chức có thể tạo một hệ thống làm việc an toàn, cho phép các nhóm nhỏ phát triển và xác thực mã nhanh chóng, độc lập, rồi triển khai an toàn tới khách hàng. Kết quả là tối đa hóa năng suất lập trình viên, học hỏi trong tổ chức, mức hài lòng cao của nhân viên và khả năng giành lợi thế trên thị trường.

Chủ đề

1. Các nguyên tắc và mẫu hình DevOps cho phép tổ chức phá vỡ xung đột cốt lõi, dai dẳng giữa Phát triển và Vận hành.
2. DevOps có thể đòi hỏi chuẩn mực văn hóa và quản trị mới, cùng những thay đổi trong thực hành kỹ thuật và kiến trúc.

Câu hỏi thảo luận

1. Chuyển đổi DevOps đòi hỏi gì từ một tổ chức? Cần những gì để tạo hệ thống làm việc an toàn?
2. Bất kể vai trò trong tổ chức là gì, một người có thể bắt đầu hình thành liên minh những người cùng chí hướng để thoát khỏi vòng xoáy đi xuống như thế nào? Làm sao thu hút lãnh đạo tham gia và ủng hộ nỗ lực DevOps?
3. Khi bắt đầu hành trình thực hiện thay đổi sâu rộng trong tổ chức, điều gì quan trọng cần ghi nhớ? Làm sao một người có thể vừa nhìn nhận sáng suốt, vừa lạc quan về rủi ro và phần thưởng của thay đổi đó?

Đọc thêm

- “Hacking văn hóa tại VelocityConf”, Jesse Robbins:
<http://www.slideshare.net/jesserobbins/hacking-culture-at-velocityconf>

Về các tác giả

Gene Kim là tác giả sách bán chạy, nhà nghiên cứu, CTO giành nhiều giải thưởng và nhà sáng lập IT Revolution. Các cuốn sách của ông gồm *The Phoenix Project*, *The Unicorn Project* và *Accelerate*. Từ năm 2014, ông là nhà sáng lập và người tổ chức DevOps Enterprise Summit, nghiên cứu các cuộc chuyển đổi công nghệ tại những tổ chức lớn, phức tạp.

Jez Humble là đồng tác giả nhiều cuốn sách bán chạy về phần mềm, trong đó có *Continuous Delivery* - tác phẩm đoạt giải Jolt - và *Accelerate* - tác phẩm đoạt giải Shingo Publication Award. Ông làm việc tại Google và giảng dạy tại Đại học California, Berkeley.

Patrick Debois là Giám đốc Quan hệ DevOps và Cố vấn tại Snyk. Ông dùng các kỹ thuật Agile trong phát triển, quản lý dự án và quản trị hệ thống để thu hẹp khoảng cách giữa dự án và vận hành.

John Willis là Giám đốc cấp cao của Văn phòng Chuyển đổi Toàn cầu tại Red Hat. Ông đã làm việc trong ngành quản lý CNTT hơn ba mươi lăm năm. Ông là đồng tác giả *Beyond The Phoenix Project* và người dẫn podcast *Profound*.

Nicole Forsgren, PhD là Partner tại Microsoft Research, lãnh đạo Developer Velocity Lab. Bà là đồng tác giả *Accelerate: The Science of Lean Software and DevOps*, cuốn sách đoạt Shingo Publication Award, và được biết đến nhiều nhất với vai trò trưởng nhóm nghiên cứu trong những nghiên cứu DevOps có quy mô lớn nhất cho đến nay. Bà từng là doanh nhân thành công - công ty được Google mua lại - giáo sư, kỹ sư hiệu suất và quản trị viên hệ thống. Công trình của bà đã được công bố trên nhiều tạp chí bình duyệt.

Bảng thuật ngữ nhanh

- **Build:** Bản dựng hoặc quá trình tạo gói phần mềm có thể kiểm thử, triển khai.
- **Brownfield:** Hiện đại hóa hệ thống đang tồn tại và đang chịu các ràng buộc di sản.
- **Continuous delivery:** Chuyển giao liên tục; phần mềm luôn ở trạng thái có thể phát hành theo yêu cầu.
- **Continuous deployment:** Triển khai liên tục; thay đổi đạt kiểm tra được tự động đưa vào production.
- **Deployment pipeline:** Chuỗi tự động đưa thay đổi qua build, kiểm thử, xác thực và triển khai.
- **Flow:** Dòng chảy của công việc từ yêu cầu đến giá trị tới tay khách hàng.
- **Greenfield:** Xây dựng hệ thống mới với ít ràng buộc di sản.
- **Infosec:** An toàn Thông tin.
- **Lead time:** Tổng thời gian từ lúc yêu cầu bắt đầu đến khi được hoàn tất/chuyển giao.
- **Production:** Môi trường thật đang phục vụ người dùng hoặc khách hàng.
- **Release:** Cho người dùng tiếp cận một phiên bản hay tính năng.
- **Telemetry:** Dữ liệu quan sát hệ thống được tự động thu thập và truyền đi, như metrics, logs và traces.
- **Trunk-based development:** Phát triển dựa trên nhánh chính với các lần tích hợp nhỏ, thường xuyên.
- **Value stream:** Toàn bộ chuỗi hoạt động cần thiết để biến nhu cầu thành giá trị cho khách hàng.

Ghi chú về bản dịch

Bản dịch này chuyển ngữ toàn bộ phần nội dung đọc được trong tệp PDF 38 trang do người dùng cung cấp. Các tiêu đề tài liệu tham khảo được dịch để người đọc hiểu nghĩa; tên tác giả, tổ chức, sản phẩm và URL được giữ nguyên. Những đoạn mang nhãn “**Chú giải của người dịch**” và bảng thuật ngữ là nội dung bổ sung nhằm làm rõ khái niệm, không thuộc nguyên văn tài liệu.